

FinShare — Master Pending Task List for Abhilash

All pending backend tasks across all phases — in priority order

April 3, 2026 | FinShare PWA | .NET 8 + SQL Server + Dapper

 **HOW TO USE THIS DOCUMENT** — Tasks are grouped by phase in priority order. Complete Phase 1 first, then Phase 2, then Phase 3 onwards. Each phase has a reference to the detailed spec document already sent to you. Tick off tasks as you complete them and confirm with Joseph.

PHASE 1 — Authentication & Security (Do First)

 **Reference:** [FinShare_ForAbhilash_AuthSecurity_v3.docx](#) | [FinShare_ForAbhilash_MASTER.docx](#)

#	Task	Type
1	ALTER TABLE Users — add EmailVerificationToken, EmailVerifyExpiry, IsEmailVerified columns	SQL
2	CREATE SP sp_SaveEmailVerifyToken	SQL
3	CREATE SP sp_VerifyEmail	SQL
4	Update Register endpoint — generate token + send verification email after registration	C#
5	Update Login endpoint — block login if IsEmailVerified = 0, return clear error message	C#
6	Add POST /api/Auth/verify-email endpoint	C#
7	Add POST /api/Auth/resend-verification endpoint	C#
8	Add PasswordValidator.cs — 8 char min, uppercase, lowercase, digit, special character	C#
9	Apply PasswordValidator in Register endpoint	C#
10	ALTER TABLE Users — add PasswordResetToken, PasswordResetExpiry columns	SQL
11	CREATE SP sp_SavePasswordResetToken	SQL
12	CREATE SP sp_ResetPassword	SQL
13	Add POST /api/Auth/forgot-password endpoint — send reset email	C#
14	Add POST /api/Auth/reset-password endpoint	C#
15	Configure email sending service (SendGrid or SMTP) for verification + reset emails	C#
16	ALTER TABLE Users — add WebAuthn credential columns (CredentialId, PublicKey, SignCount, AaGuid)	SQL
17	Install Fido2NetLib NuGet package	C#
18	Add POST /api/Auth/webauthn/register-options endpoint	C#
19	Add POST /api/Auth/webauthn/register-verify endpoint	C#
20	Add POST /api/Auth/webauthn/login-options endpoint	C#
21	Add POST /api/Auth/webauthn/login-verify endpoint	C#

PHASE 2 — Security Hardening (Do Second)

 **Reference:** [FinShare_ForAbhilash_SecurityPhase2.docx](#)

 **IMPORTANT BEFORE STARTING PHASE 2** — HTTPS must already be active on fintrack.sysdatacenter.com. JWT secret must be stored in environment variables on the server — NOT in appsettings.json or source code.

#	Task	Type
1	Add builder.Services.AddRateLimiter() in Program.cs — login endpoint: 5 attempts per 15 min per IP	C#
2	Add builder.Services.AddRateLimiter() — register endpoint: 3 per hour per IP	C#
3	Add app.UseForwardedHeaders() + GetClientIP() helper to read correct IP behind proxy	C#
4	ALTER TABLE Users — add LoginAttempts (INT), LockoutUntil (DATETIME NULL) columns	SQL
5	CREATE SP sp_RecordFailedLogin — increment counter, set lockout after 5 fails (15 min)	SQL
6	CREATE SP sp_ResetLoginAttempts — called on successful login	SQL
7	Update Login endpoint — check lockout, call sp_RecordFailedLogin on fail, sp_ResetLoginAttempts on success	C#
8	Verify BCrypt is using minimum 10 rounds — confirm in Register and Reset Password endpoints	C#
9	CREATE SP sp_CleanupExpiredTokens — delete expired email verify + password reset tokens	SQL
10	Set up SQL Agent job OR .NET BackgroundService to run sp_CleanupExpiredTokens daily	Both
11	CREATE TABLE AuditLogs — EventType, MemberId, IPAddress, Details, CreatedAt	SQL
12	CREATE SP sp_WriteAuditLog	SQL
13	Add audit log calls in Login, Register, ForgotPassword, ResetPassword, Lockout events	C#
14	Plan 90-day audit log retention — archive or delete older rows (discuss with Joseph)	SQL
15	CREATE TABLE UserRefreshTokens — Token, MemberId, ExpiresAt, IsRevoked, CreatedAt	SQL
16	CREATE SP sp_SaveRefreshToken	SQL
17	CREATE SP sp_ValidateRefreshToken	SQL
18	CREATE SP sp_RevokeRefreshToken	SQL
19	CREATE SP sp_RevokeAllUserRefreshTokens	SQL
20	Add POST /api/Auth/refresh endpoint — validate refresh token cookie, issue new access + refresh token pair	C#
21	Add POST /api/Auth/revoke endpoint — revoke current refresh token on logout	C#
22	Update Login endpoint — set refresh token as httpOnly cookie in response	C#
23	Store JWT secret in server environment variable — confirm NOT in appsettings.json	Config

PHASE 3 — New Features: Profile, Transactions, Notifications (Do Third)

 [Reference: FinShare_ForAbhilash_NewFeatures.docx](#)

#	Task	Type
1	ALTER TABLE Users — add ProfilePhotoUrl NVARCHAR(500) NULL column	SQL
2	CREATE SP sp_GetUserProfile	SQL
3	CREATE SP sp_UpdateUserProfile	SQL
4	CREATE SP sp_UpdateProfilePhoto	SQL
5	Add GET /api/User/profile endpoint	C#
6	Add PUT /api/User/profile endpoint	C#
7	Install SixLabors.ImageSharp NuGet package	C#
8	Add POST /api/User/profile/photo endpoint — validate image type + size, resize to 200x200, save to /uploads/avatars/	C#
9	CREATE SP sp_ChangePassword	SQL

10	Add PUT /api/User/change-password endpoint — verify current password with BCrypt, validate new with PasswordValidator	C#
11	CREATE SP sp_GetTransactionHistory — UNION ALL of group expenses, personal expenses, settlements with filters + pagination	SQL
12	CREATE SP sp_GetTransactionSummary — return TotalSpent, TotalToReceive, TotalToPay	SQL
13	Add GET /api/Transaction/history endpoint — accepts from, to, type, groupId, category, page, pageSize query params	C#
14	CREATE TABLE Notifications — NotificationId, MemberId, Title, Body, Type, ReferenceId, ReferenceType, IsRead, CreatedAt	SQL
15	CREATE SP sp_CreateNotification	SQL
16	CREATE SP sp_GetNotifications — returns UnreadCount + paginated list	SQL
17	CREATE SP sp_MarkNotificationRead	SQL
18	CREATE SP sp_MarkAllNotificationsRead	SQL
19	Add GET /api/Notifications endpoint	C#
20	Add PUT /api/Notifications/{id}/read endpoint	C#
21	Add PUT /api/Notifications/read-all endpoint	C#
22	Add sp_CreateNotification calls inside: AddExpense, EditExpense, Settlement, AddGroupMember, ChangePassword controllers	C#

PHASE 4 — Bank Details (Do Fourth)

 **Reference: FinShare_ForAbhilash_BankDetails.docx**

#	Task	Type
1	CREATE TABLE MemberBankDetails — BankDetailId, MemberId, AccountHolderName, IBANEncrypted, BankName, Branch, CreatedAt, UpdatedAt	SQL
2	CREATE UNIQUE INDEX UX_MemberBankDetails_MemberId on MemberBankDetails(MemberId)	SQL
3	Create EncryptionService.cs — AES-256 Encrypt(), Decrypt(), MaskIBAN() methods	C#
4	Register EncryptionService as Singleton in Program.cs	C#
5	Generate strong Encryption:Key (32 chars) and Encryption:IV (16 chars) — store in server environment variables only	Config
6	CREATE SP sp_SaveBankDetails (MERGE upsert)	SQL
7	CREATE SP sp_GetBankDetails	SQL
8	CREATE SP sp_GetMemberBankDetailsForSettlement — verify requesting member is the payer of that settlement before returning	SQL
9	CREATE SP sp_DeleteBankDetails	SQL
10	Add GET /api/User/bank-details endpoint — decrypt IBAN, return full + masked	C#
11	Add PUT /api/User/bank-details endpoint — encrypt IBAN before saving	C#
12	Add DELETE /api/User/bank-details endpoint	C#
13	Add GET /api/Settlement/{settlementId}/payee-bank-details endpoint — 403 if not the payer	C#
14	Add SaveBankDetailsRequest model with IBAN regex validation	C#
15	Add audit log entries for BANK_DETAILS_SAVED, BANK_DETAILS_ACCESSED, BANK_DETAILS_DELETED	C#

PHASE 5 — Group Events Backend (Do After Phase 2 Confirmed)

 **Reference: FinShare_ForAbhilash_EventBackend.docx**

i NOTE — The Events feature is already working on the frontend using localStorage. This phase moves it to the real database so all group members can see each other's events. Start only after Joseph confirms Phase 2 is complete.

#	Task	Type
1	CREATE TABLE GroupEvents — EventId, GroupId, CreatedBy, Title, EventDate, EventTime, Location, Notes, CreatedAt, UpdatedAt	SQL
2	CREATE INDEX IX_GroupEvents_GroupId_EventDate on GroupEvents(GroupId, EventDate)	SQL
3	CREATE SP sp_GetGroupEvents — verify member belongs to group before returning	SQL
4	CREATE SP sp_CreateGroupEvent — verify creator is a group member	SQL
5	CREATE SP sp_UpdateGroupEvent — creator-only check	SQL
6	CREATE SP sp_DeleteGroupEvent — creator-only check	SQL
7	Create GroupEventsController.cs with all 4 endpoints (GET, POST, PUT, DELETE)	C#
8	Add CreateGroupEventRequest model with validation	C#

TOTAL TASK COUNT

Phase	Description	Tasks
Phase 1	Authentication & Security	21
Phase 2	Security Hardening	23
Phase 3	Profile, Transactions, Notifications	22
Phase 4	Bank Details	15
Phase 5	Group Events Backend	8
TOTAL		89

— Joseph Xavier / FinShare Project / April 3, 2026 / All detailed specs sent separately per phase